
Modelización Avanzada en Logística y Transporte

Unidad 2: Bases de programación matemática y teoría de grafos

Luis M. Torres

Escuela Politécnica del Litoral
Guayaquil, Octubre 2006

Contenido

- Programas lineales
- Programas enteros y mixtos
- Optimización en grafos y redes

Contenido

- Programas lineales
 - Motivación
 - Definición y propiedades básicas
 - Solución
 - Herramientas de software
- Programas enteros y mixtos
- Optimización en grafos y redes

Programas lineales – Motivación

Problema:

Industria química produce 3 tipos de detergente: A, B, C

Emplea dos ingredientes activos: I1, I2

Datos: composiciones, precios de venta, disp. de ingredientes y volúmenes máximos de producción

Det. [1 kg]	I1 [kg]	I2 [kg]	P.V. [USD]	Máx [kg]
A	0.3	0.6	1.5	300
B	0.4	0.5	1.7	250
C	0.6	0.3	1.8	200

Disp.	I1 [kg]	I2 [kg]
	300	320

Programas lineales – Motivación

Problema:

Industria química produce 3 tipos de detergente: A, B, C

Emplea dos ingredientes activos: I1, I2

Datos: composiciones, precios de venta, disp. de ingredientes y volúmenes máximos de producción

Cuál es la cantidad óptima a producir de cada detergente?

B	0.4	0.5	1.7	250
C	0.6	0.3	1.8	200

Disp.	I1 [kg]	I2 [kg]
	300	320

Programas lineales – Motivación

Variables:

x_A , x_B , x_C : cantidades a producir

Utilidad (a maximizar):

$$1.5x_A + 1.7x_B + 1.8x_C$$

Disp. ingredientes:

$$I1: 0.3x_A + 0.4x_B + 0.6x_C \leq 300$$

$$I2: 0.6x_A + 0.5x_B + 0.3x_C \leq 320$$

Vol. máximos de producción:

$$x_A \leq 300, \quad x_B \leq 250, \quad x_C \leq 200$$

No negatividad:

$$x_A \geq 0, \quad x_B \geq 0, \quad x_C \geq 0$$

Programas lineales – Motivación

Obtenemos el **programa lineal**:

$$\max 1.5x_A + 1.7x_B + 1.8x_C$$

s.r.

$$0.3x_A + 0.4x_B + 0.6x_C \leq 300$$

$$0.6x_A + 0.5x_B + 0.3x_C \leq 320$$

$$0 \leq x_A \leq 300$$

$$0 \leq x_B \leq 250$$

$$0 \leq x_C \leq 200$$

Programas lineales – Motivación

Obtenemos el **programa lineal**:

$$\max 1.5x_A + 1.7x_B + 1.8x_C$$

Func. objetivo (lineal)

S.r.

$$0.3x_A + 0.4x_B + 0.6x_C \leq 300$$

$$0.6x_A + 0.5x_B + 0.3x_C \leq 320$$

$$0 \leq x_A \leq 300$$

$$0 \leq x_B \leq 250$$

$$0 \leq x_C \leq 200$$

Restricciones lineales

Programas lineales

Definición:

Un programa lineal consiste en maximizar (minimizar) una función lineal sobre un dominio factible dado por ecuaciones y desigualdades lineales.

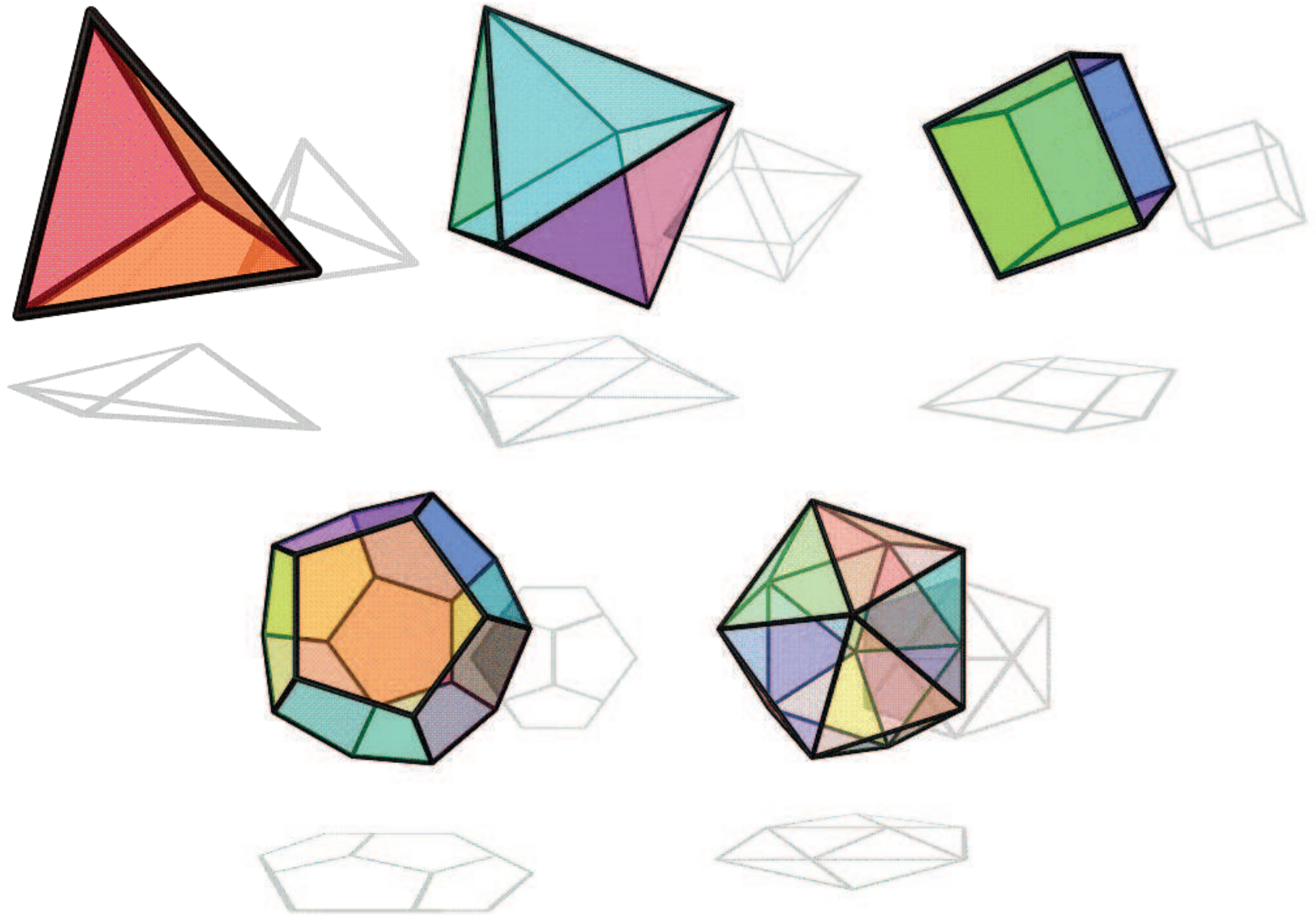
$$\begin{array}{ll}\max & c^T x \\ \text{s.r.} & \\ & Ax \leq b \\ & \text{(Canónica)}\end{array}$$

$$\begin{array}{ll}\max & c^T x \\ \text{s.r.} & \\ & Ax = b \\ & x \geq 0 \\ & \text{(Estándar)}\end{array}$$

Con $x, c \in \mathbb{R}^n$, $b \in \mathbb{R}^m$ y $A \in \mathbb{R}^{m \times n}$

Propiedades

El conjunto de soluciones factibles define un **poliedro**



Propiedades

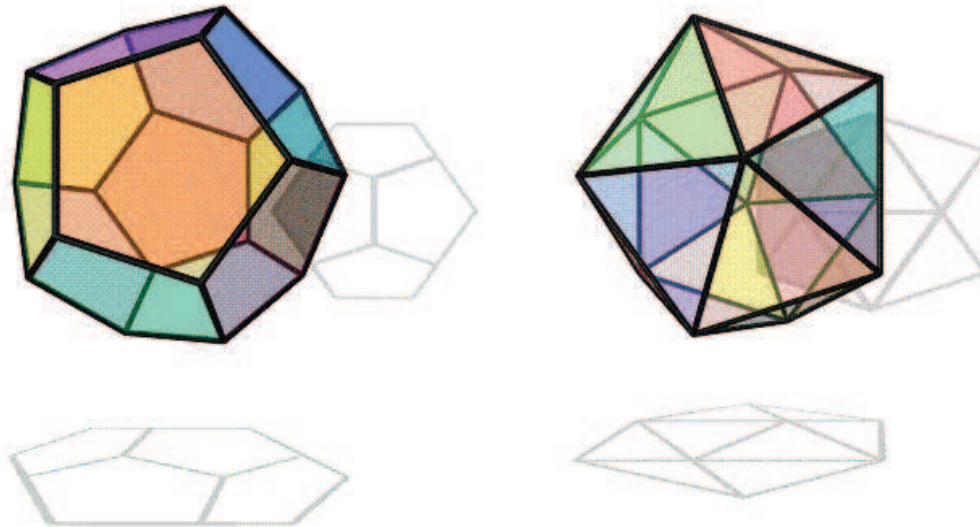
El conjunto de soluciones factibles define un **poliedro**



Observación:

Curvas de nivel de una función lineal $\equiv$ hiperplanos

$\leadsto$ Un óptimo del PL se alcanza en un vértice



Propiedades

- Todo programa lineal tiene asociado un **programa dual**

$$\max c^T x$$

s.r.

$$Ax = b$$

$$x \in \mathbb{R}^n, x \geq 0$$

(primal)

$$\min y^T b$$

s.r.

$$y^T A \geq c^T$$

$$y \in \mathbb{R}^m$$

(dual)

- Si uno de los programas no tiene solución factible, el otro es no acotado
- Caso contrario, los óptimos coinciden (T. Dualidad)

Solución de PLs

Algoritmo del simplex

Propuesto originalmente por
George Dantzig (1940s)
Aún el más usado en la actualidad
(con modificaciones)
Resuelve PLs en la forma estándar

Idea: solución básica

$A = (B, N)$, B de rango completo

$$x^T = (x_B^T, 0), \quad Bx_B = b, \quad x_B \geq 0$$

Calcular $y \in \mathbb{R}^m : y^T B = c_B^T$

Si $y^T N \geq c_N^T \Rightarrow x$ es **solución óptima!!!**



Método del simplex

1. Construir solución básica inicial x_B
2. Resolver $y^T B = c_B^T$ (precios duales)
3. Si $y^T N \geq c_N^T$, FIN (sol. óptima)
4. Elegir $j \in N : y_j^T A_j < c_j^T$ (var. entrante)
5. Aumentar x_j manteniendo
$$x_B = B^{-1}(b - N x_N) = B^{-1}(b - A_j x_j), \quad x_B \geq 0$$
6. Si $x_j \rightarrow +\infty$, FIN (prob. no acotado)
7. Elegir $i \in B : x_i = 0$ para el menor valor de x_j (var. saliente)
8. Actualizar B (pivotaje)
9. Ir a 2

Método del simplex

Ejemplo:
(Ver [demo](#))

$$\max 1.5x_A + 1.7x_B + 1.8x_C$$

s.r.

$$0.3x_A + 0.4x_B + 0.6x_C \leq 300$$

$$0.6x_A + 0.5x_B + 0.3x_C \leq 320$$

$$0 \leq x_A \leq 300$$

$$0 \leq x_B \leq 250$$

$$0 \leq x_C \leq 200$$

Método del simplex

Ejemplo:
(Ver [demo](#))

$$\max 1.5x_A + 1.7x_B + 1.8x_C$$

s.r.

$$0.3x_A + 0.4x_B + 0.6x_C \leq 300$$

$$0.6x_A + 0.5x_B + 0.3x_C \leq 320$$

$$0 \leq x_A \leq 300$$

$$0 \leq x_B \leq 250$$

$$0 \leq x_C \leq 200$$

Sol. óptima:

$$x_A = 225.00$$

$$x_B = 250.00$$

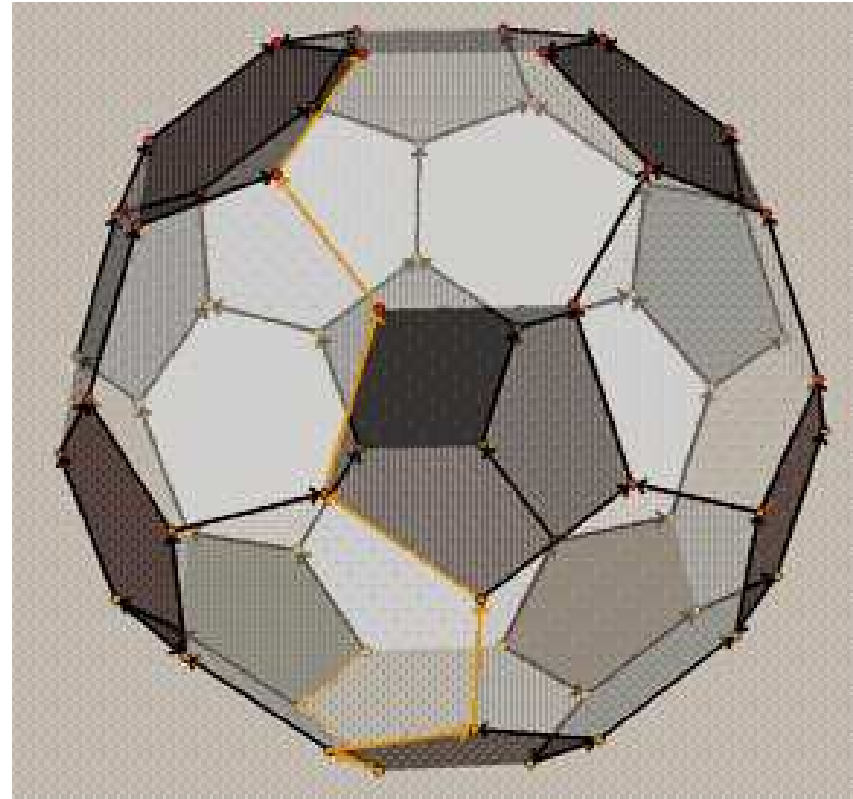
$$x_C = 200.00$$

$$z = 1122.50$$

Interpretación geométrica

Cada solución básica está asociada a un vértice del poliedro.

El algoritmo del simplex construye una ruta desde el vértice inicial hasta el vértice óptimo.



(Autor: Marc Pfetsch)

Otros métodos de solución

- El algoritmo del simplex ha sido mejorado sustancialmente en las últimas seis décadas

$$\max c^T x$$

$$Ax = b$$

$$Cx \leq d$$

$$\ell \leq x \leq u$$

- Desarrollo de estructuras de datos
- Desarrollo de hardware
- Método del elipsoide (Leonid Khachiyan, 1979)
- Métodos de punto interior (N. Karmarkar, 1984)

Herramientas de software

Solvers de LPs

- CPLEX (ILOG)
- Xpress-MP (Dash)
- CLP (COIN-OR)
- SoPlex (ZIB)
- GLPK (GNU)
- LINDO
- Excel Solver
- ...

Lenguajes de modelamiento

- GAMS
- AMPL
- ZIMPL

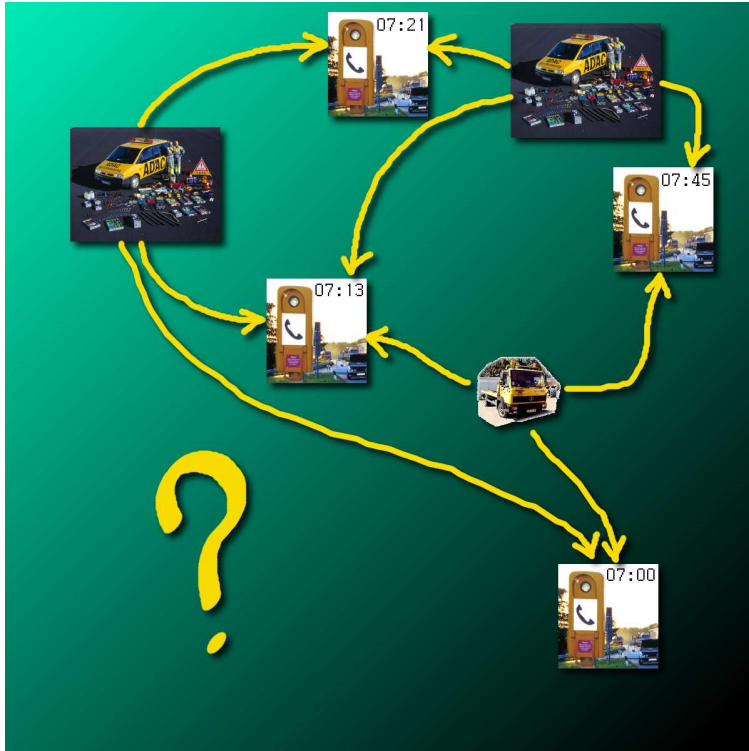
Herramientas de software

- La mayoría de solvers admiten LP's
 - como archivos de texto
 - interactivamente desde el teclado
 - a través de API's de C/C++ ...
- Todos los solvers implementan variantes del método del simplex
- Algunos solvers usan adicionalmente métodos de punto interior
- Los lenguajes de modelamiento permiten escribir familias de restricciones de manera abstracta

Contenido

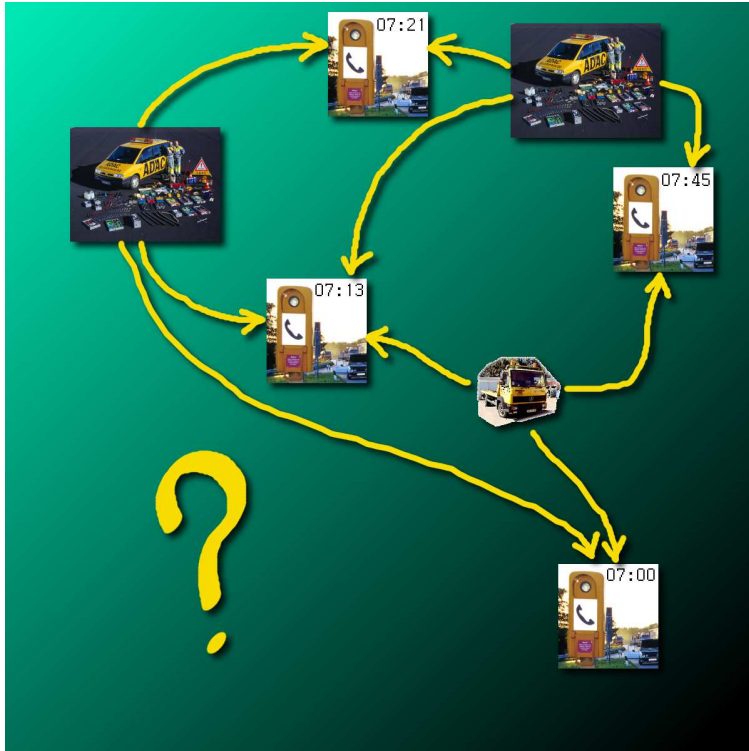
- Programas lineales
- Programas enteros y mixtos
 - Motivación y definición
 - Propiedades básicas
 - Solución
 - Herramientas de software
- Optimización en grafos y redes

El problema del ADAC

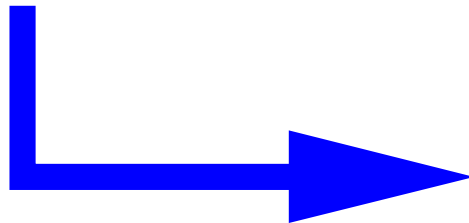


pedidos, unidades,
contratistas...

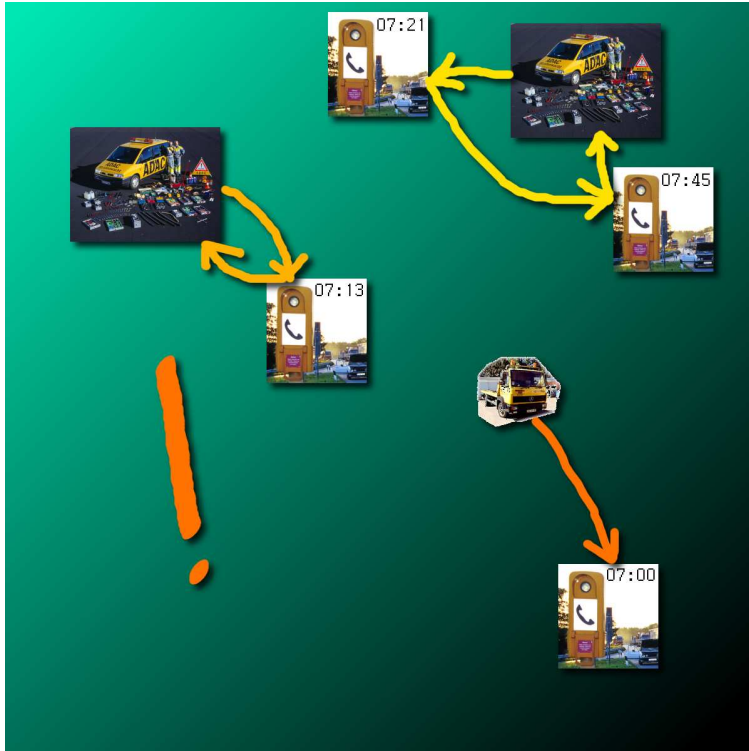
El problema del ADAC



Central de despacho



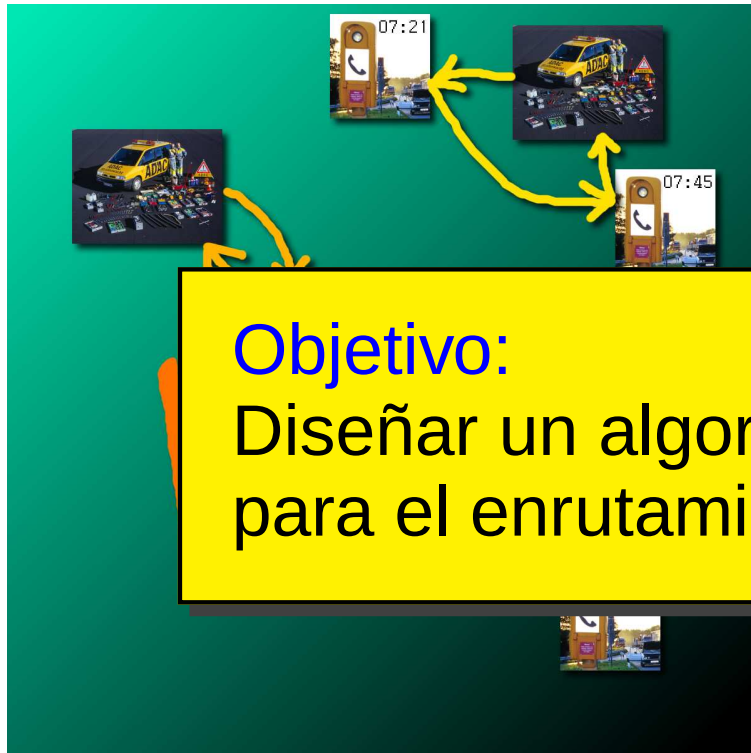
El problema del ADAC



Plan de rutas



El problema del ADAC



Objetivo:

Diseñar un algoritmo de optimización para el enrutamiento de las unidades

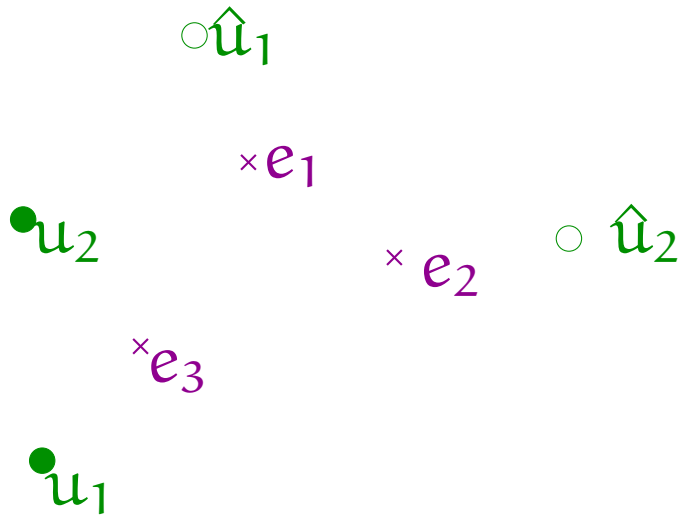
Plan de rutas



Formulación del modelo

Idea:

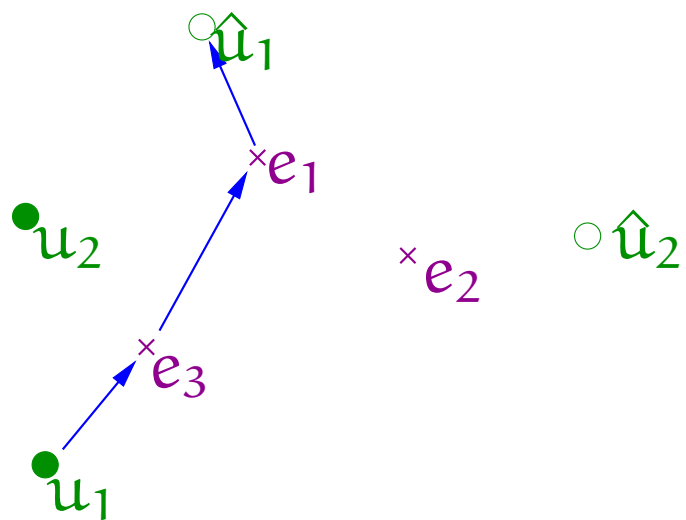
Definir una variable binaria por cada **ruta** de servicio.



Formulación del modelo

Idea:

Definir una variable binaria por cada **ruta** de servicio.

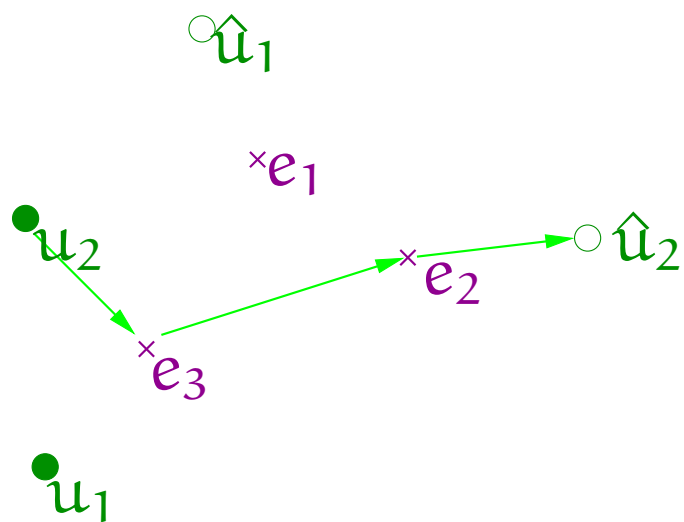


T_1	
1	r_1
0	r_2
1	r_3
1	u_1
0	u_2
c_{T_1}	
x_{T_1}	

Formulación del modelo

Idea:

Definir una variable binaria por cada **ruta** de servicio.

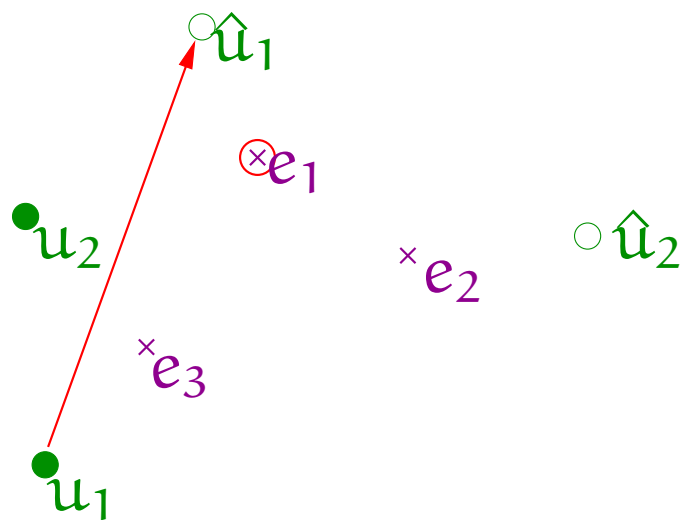


	T_1	T_2	
	1	0	r_1
	0	1	r_2
	1	1	r_3
	1	0	u_1
	0	1	u_2
	c_{T_1}	c_{T_2}	
	x_{T_1}	x_{T_2}	

Formulación del modelo

Idea:

Definir una variable binaria por cada **ruta** de servicio.

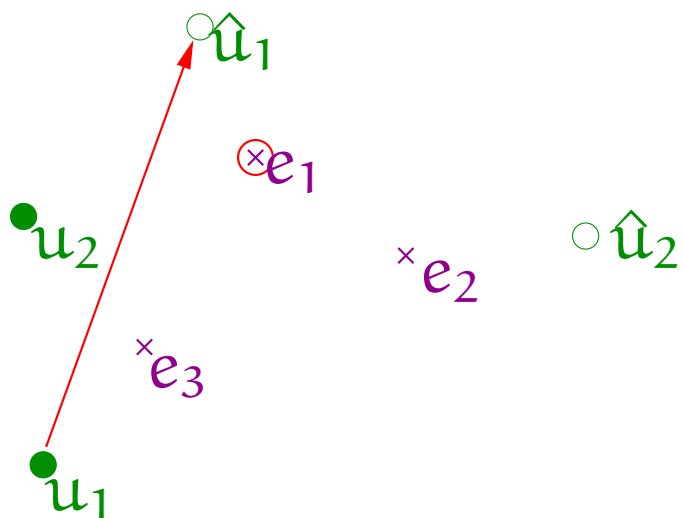


	T_1	T_2	T_3	T_4	
	1	0	0	1	r_1
	0	1	0	0	r_2
	1	1	0	0	r_3
	1	0	1	0	u_1
	0	1	0	0	u_2
	c_{T_1}	c_{T_2}	c_{T_3}	c_{T_4}	
	x_{T_1}	x_{T_2}	x_{T_3}	x_{T_4}	

Formulación del modelo

Idea:

Definir una variable binaria por cada **ruta** de servicio.



	T_1	T_2	T_3	T_4	...	T_N	
A	1	0	0	1	...		r_1
	0	1	0	0	...		r_2
	1	1	0	0	...		r_3
	1	0	1	0	...		u_1
	0	1	0	0	...		u_2
c^T	c_{T_1}	c_{T_2}	c_{T_3}	c_{T_4}	...	c_{T_N}	
x^T	x_{T_1}	x_{T_2}	x_{T_3}	x_{T_4}	...	x_{T_N}	

Formulación del modelo

Idea:

Definir una variable binaria por cada **ruta** de servicio.

Formular un **enorme**
Problema de
Particionamiento
(IP):

$$\min c^T x$$

s.t.

$$Ax = 1$$

$$x \in \{0, 1\}^N$$

r_1

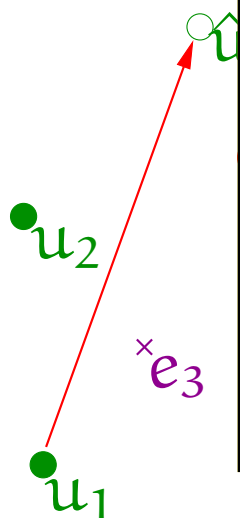
r_2

r_3

u_1

u_2

c^T	c_{T_1}	c_{T_2}	c_{T_3}	c_{T_4}	$\dots$	c_{T_N}
x^T	x_{T_1}	x_{T_2}	x_{T_3}	x_{T_4}	$\dots$	x_{T_N}



Programas enteros mixtos

Definición:

Un **programa entero** (integer program, IP) es un PL donde las variables deben además cumplir la condición de ser enteras

$$\max c^T x$$

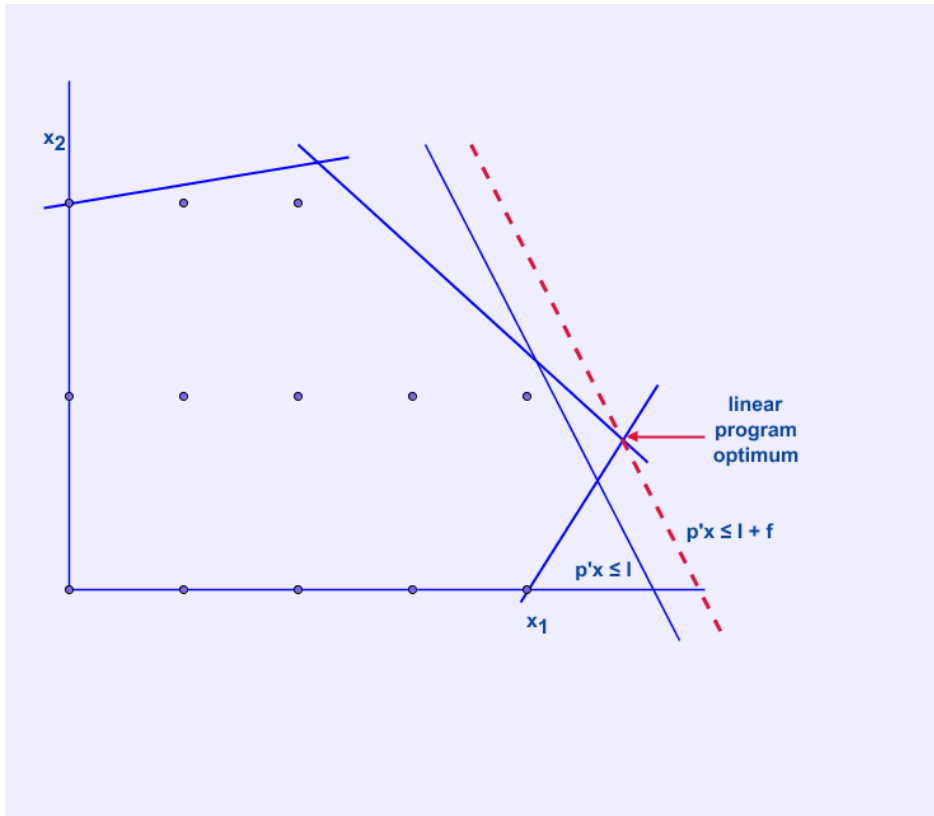
s.r.

$$Ax = b,$$

$$x \geq 0, \quad x \in \mathbb{Z}^n$$

Un PL donde ciertas variables son enteras y otras son reales se conoce como **programa entero mixto** (mixed integer program, MIP)

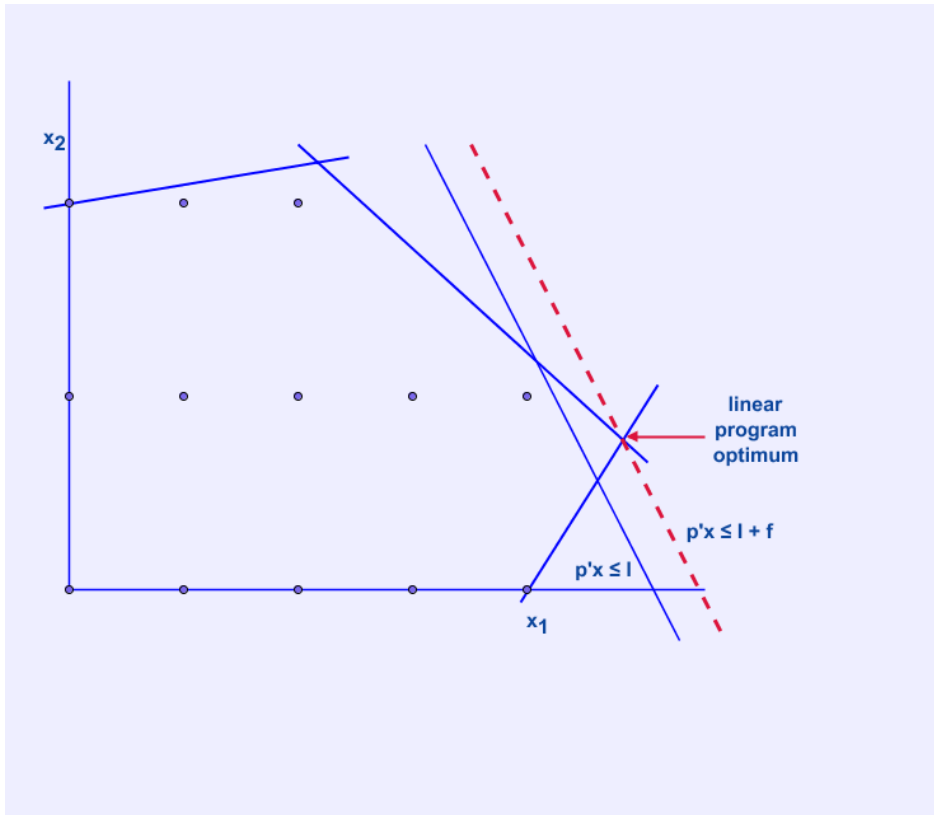
Propiedades



La región factible de un IP es el conjunto de puntos contenidos en el interior de un poliedro P .

Optimizar sobre P equivale a resolver la relajación lineal del IP y proporciona una cota para el óptimo.

Propiedades



La región factible de un IP es el conjunto de puntos contenidos en el interior de un poliedro P .

Optimizar sobre P equivale a resolver la relajación lineal del IP y proporciona una cota para el óptimo.

Resolver un IP en general es difícil!!!
No existe un algoritmo equivalente al simplex

Solución de MIPs

Branch-and-Bound

Técnica “inteligente” de enumeración

$$\max 0.2x_1 + 0.3x_2 + 0.5x_3 + 0.1x_4$$

s.r.

$$0.5x_1 + 1.0x_2 + 1.5x_3 + 0.1x_4 \leq 3.1$$

$$0.3x_1 + 0.8x_2 + 1.5x_3 + 0.4x_4 \leq 2.5$$

$$0.2x_1 + 0.2x_2 + 0.3x_3 + 0.1x_4 \leq 0.4$$

$$x_j \in \{0, 1\}, \quad \forall j = 1, \dots, 4$$

Resolviendo la relajación lineal, obtenemos el óptimo:

$$x_1 = x_4 = 0, \quad x_3 = 1, \quad x_2 = 0.5, \quad z = 0.65$$

Branch-and-Bound (2)

Ahora conocemos que el valor del óptimo está acotado por 0.65 (“**bound**”)

Sustituyendo x_2 por 0 o 1, generamos dos problemas nuevos, más pequeños: (“**branch**”)

$$\max 0.2x_1 + 0.5x_3 + 0.1x_4$$

s.r.

$$0.5x_1 + 1.5x_3 + 0.1x_4 \leq 3.1$$

$$0.3x_1 + 1.5x_3 + 0.4x_4 \leq 2.5$$

$$0.2x_1 + 0.3x_3 + 0.1x_4 \leq 0.4$$

$$x_j \in \{0, 1\}, \quad \forall j = 1, \dots, 4$$

$$\max 0.2x_1 + 0.5x_3 + 0.1x_4$$

s.r.

$$0.5x_1 + 1.5x_3 + 0.1x_4 \leq 2.1$$

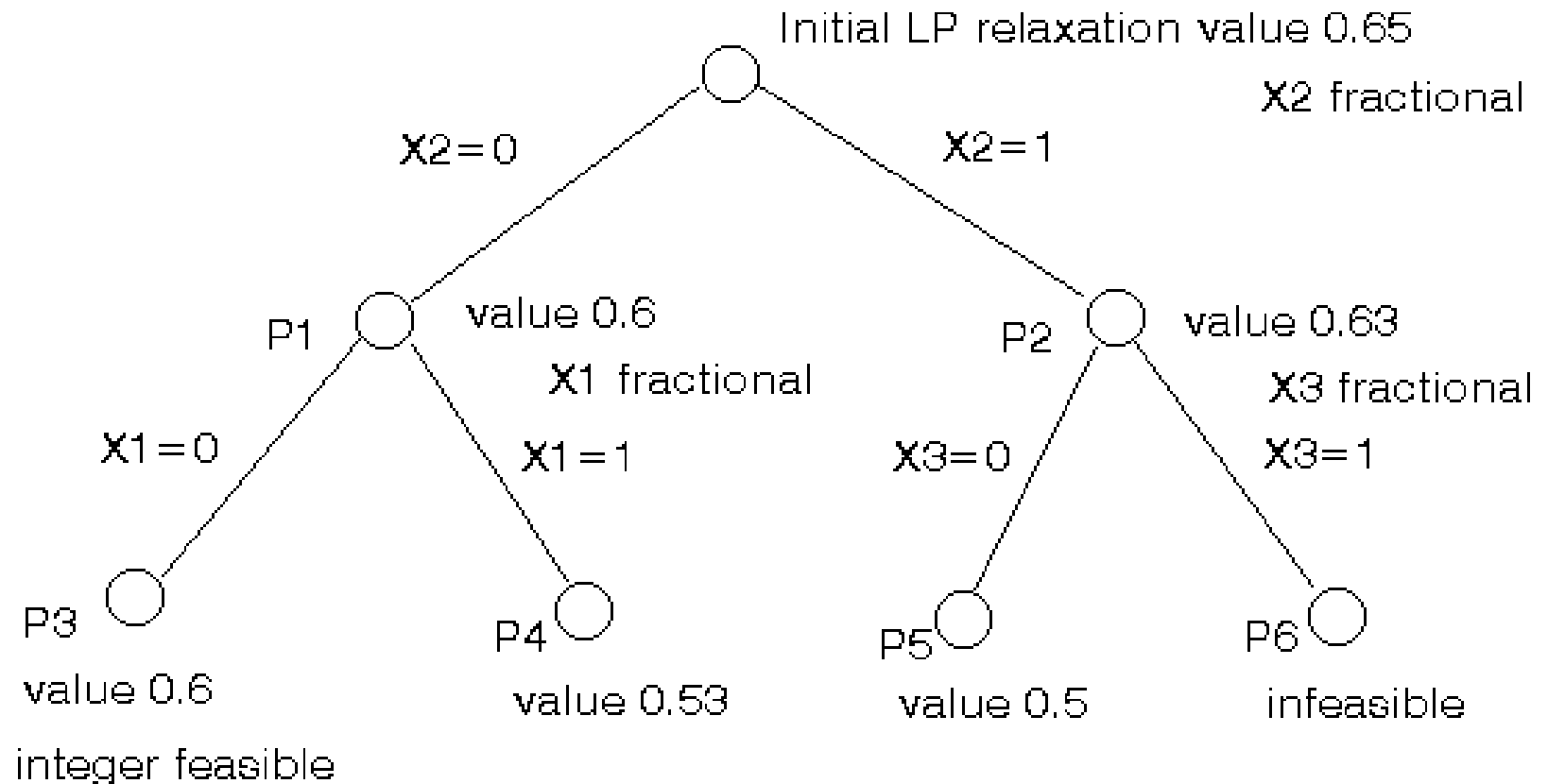
$$0.3x_1 + 1.5x_3 + 0.4x_4 \leq 1.7$$

$$0.2x_1 + 0.3x_3 + 0.1x_4 \leq 0.2$$

$$x_j \in \{0, 1\}, \quad \forall j = 1, \dots, 4$$

Branch-and-Bound (3)

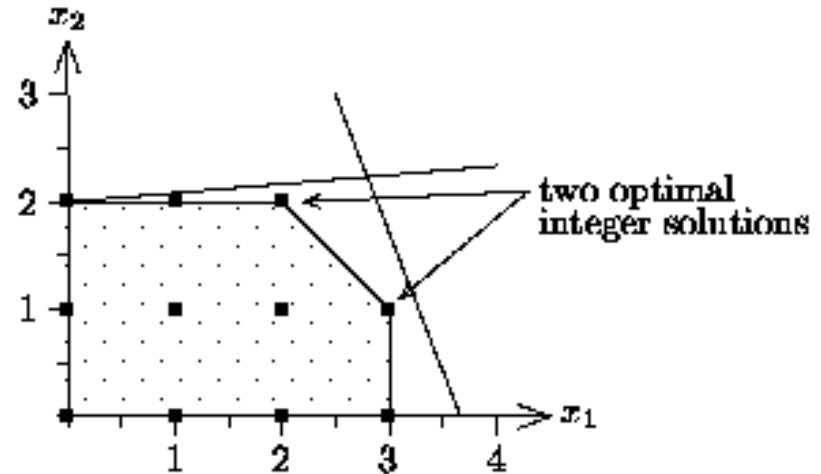
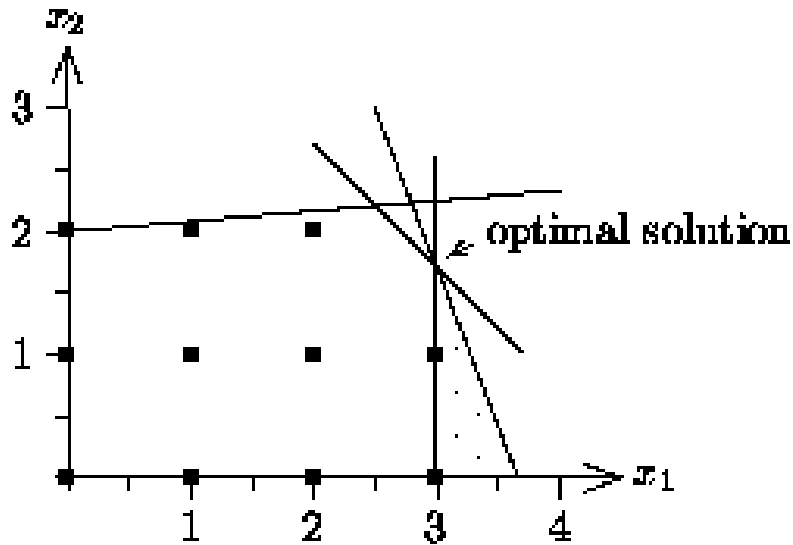
Continuamos en un árbol de búsqueda



Las cotas de los PLs aceleran la búsqueda (**tree pruning**)

Planos cortantes

Se usan para mejorar las cotas de los LPs



Observación:

La condición de integralidad puede sustituirse por
(muchísimas) desigualdades lineales

Herramientas de software

Solvers de MIPs

- CPLEX (ILOG)
- Xpress-MP (Dash)
- CBC (COIN-OR)
- SCIP (ZIB)
- GLPK (GNU)
- ...

Combinan

- Branch-and-Bound
- Cutting planes
- Branch-and-Cut
- Heurísticas
- ...

Contenido

- Programas lineales
- Programas enteros y mixtos
- Optimización en grafos y redes
 - Motivación
 - Propiedades básicas
 - Problemas clásicos

Todo empezó en Königsberg...



Kaliningrado (2005)
exclave ruso en el mar
báltico...

Todo empezó en Königsberg...



Kaliningrado (2005)
exclave ruso en el mar
báltico...

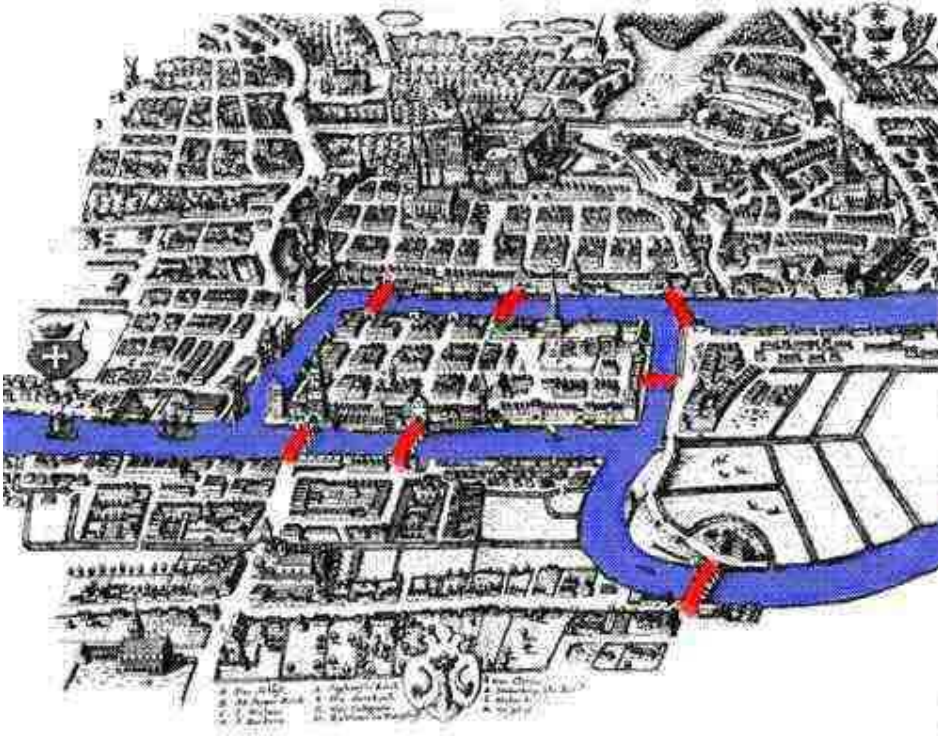
Königsberg (~ 1750)
... puerto del antiguo
imperio prusiano

...con un problema de puentes



El Pregel atraviesa la ciudad formando dos islas

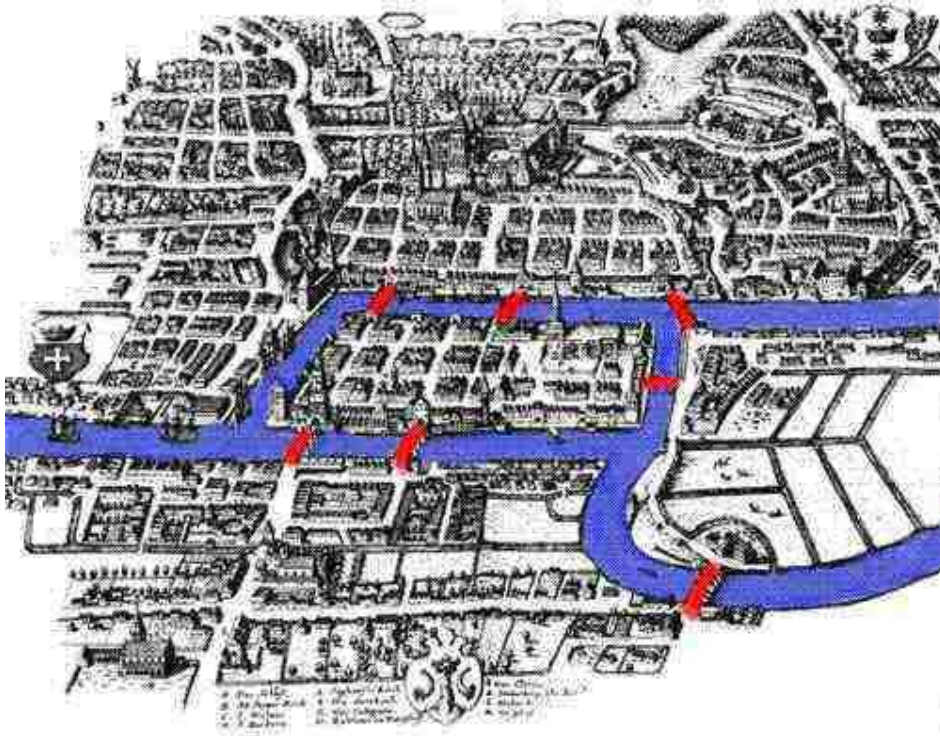
...con un problema de puentes



El Pregel atraviesa la ciudad formando dos islas

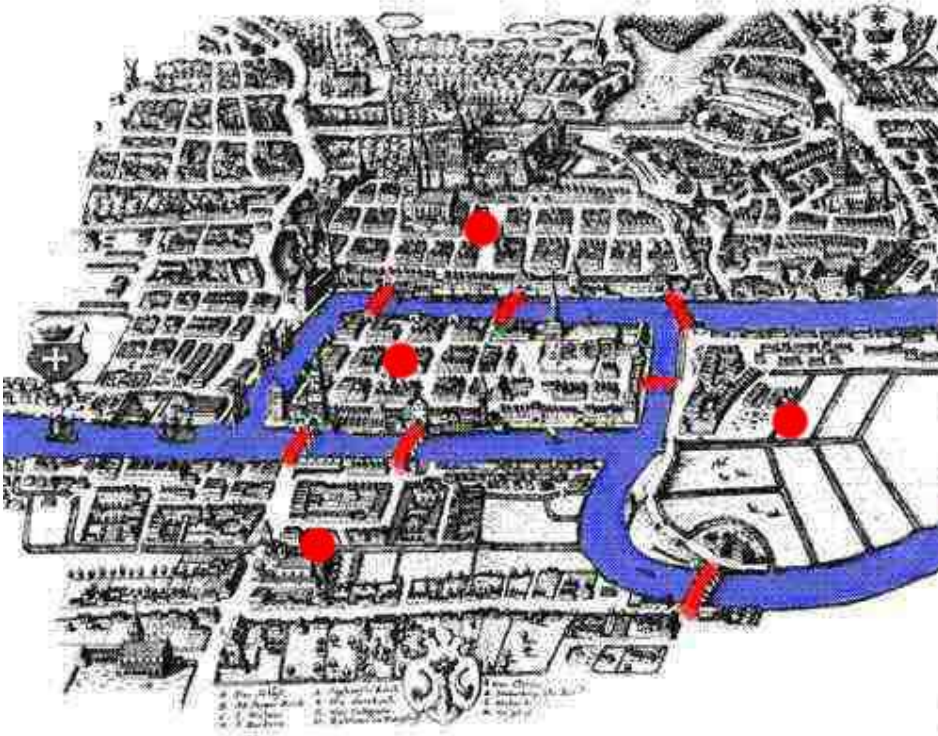
Problema:
Cruzar cada uno de los siete puentes exactamente una vez

Modelo

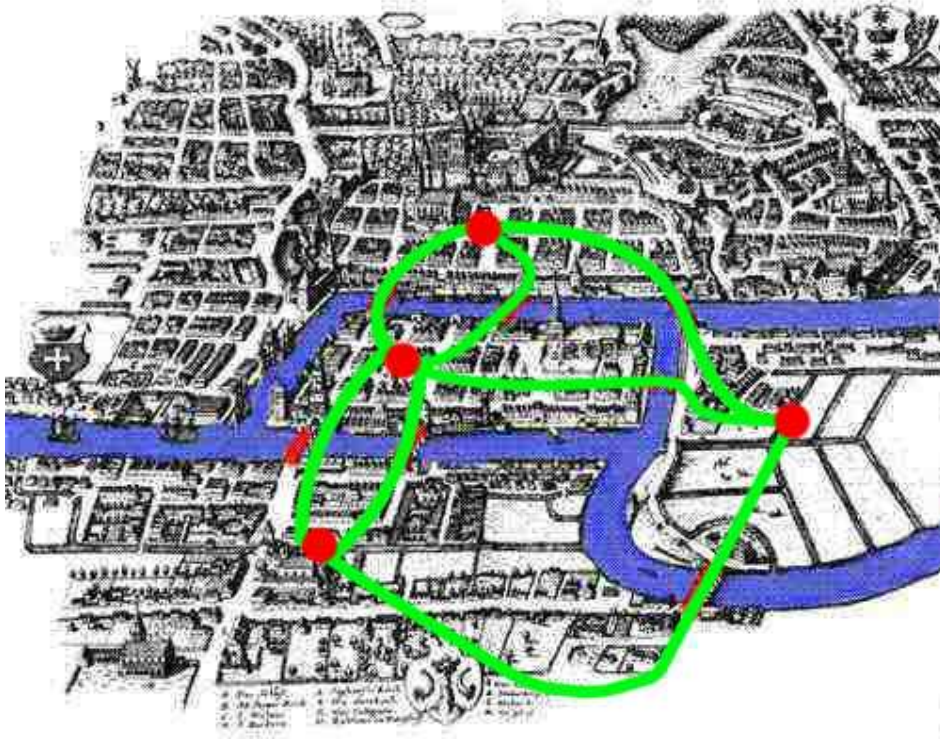


Modelo

Cuatro orillas...



Modelo

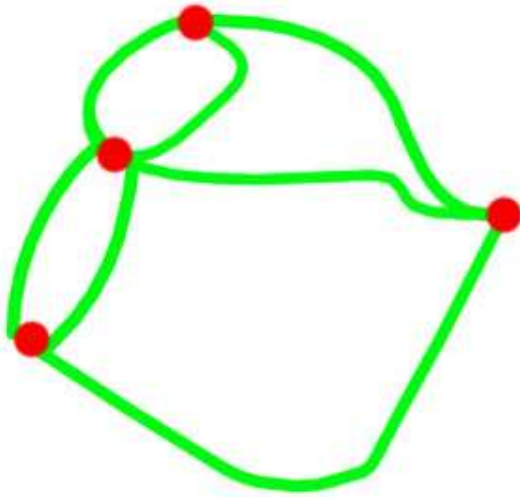


Cuatro orillas...

... unidas por siete
puentes ...

Modelo

Cuatro orillas...



... unidas por siete
puentes ...

Idea:
Olvidar la ciudad,
retener la estructura!

Grafos

Definición:

Un grafo es un par ordenado $G = (V, E)$ donde:

- V es un conjunto **finito**
- E es un **multiconjunto** de la forma
 $E \subseteq \{\{i, j\} : i \in V, j \in V\}$

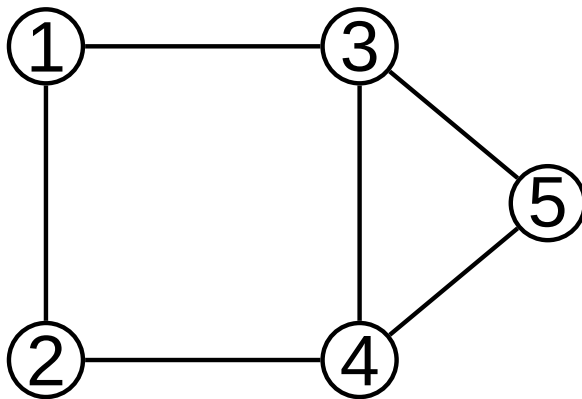
Grafos

Definición:

Un grafo es un par ordenado $G = (V, E)$ donde:

- V es un conjunto **finito**
- E es un **multiconjunto** de la forma
 $E \subseteq \{\{i, j\} : i \in V, j \in V\}$

Ejemplo:



$$V = \{1, 2, 3, 4, 5\}$$

$$E = \{\{1, 2\}, \{1, 3\}, \{2, 4\}, \\ \{3, 4\}, \{3, 5\}, \{4, 5\}\}$$

Grafos

Definición:

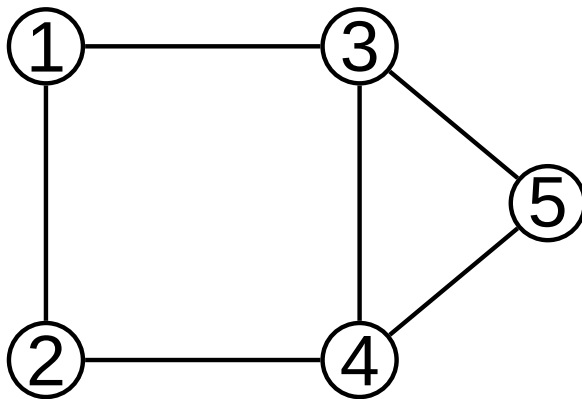
Un grafo es un par ordenado $G = (V, E)$ donde:

- V es un conjunto **finito**
- E es un **multiconjunto** de la forma

E

Los elementos de V se llaman **nodos** de G ,
los elementos de E son las **aristas** de G .

Ejemp



$$V = \{1, 2, 3, 4, 5\}$$

$$E = \{\{1, 2\}, \{1, 3\}, \{2, 4\}, \\ \{3, 4\}, \{3, 5\}, \{4, 5\}\}$$

Algunos conceptos básicos

- una arista $e \in E$ es **incidente** a un nodo $i \in V$ si $i \in e$

Algunos conceptos básicos

- una arista $e \in E$ es **incidente** a un nodo $i \in V$ si $i \in e$
- dos nodos $i, j \in V$ son **adyacentes** si $\{i, j\} \in E$

Algunos conceptos básicos

- una arista $e \in E$ es **incidente** a un nodo $i \in V$ si $i \in e$
- dos nodos $i, j \in V$ son **adyacentes** si $\{i, j\} \in E$
- el conjunto de nodos adyacentes a un cierto nodo $i \in V$ es la **vecindad** de i :

$$N(i) := \{j \in V : ij \in E\}$$

Algunos conceptos básicos

- una arista $e \in E$ es **incidente** a un nodo $i \in V$ si $i \in e$
- dos nodos $i, j \in V$ son **adyacentes** si $\{i, j\} \in E$
- el conjunto de nodos adyacentes a un cierto nodo $i \in V$ es la **vecindad** de i :

$$N(i) := \{j \in V : ij \in E\}$$

- el **grado** de un nodo i es la cardinalidad de su vecindad:

$$d(i) := |N(i)|$$

... y coincide con el número de aristas incidentes a i

Representación computacional

Matriz de adyacencia:

Dado un grafo $G = (V, E)$, definimos su matriz de adyacencia $M(G) \in \mathcal{M}_{V \times V}$ por medio de:

$$m_{ij} := \begin{cases} 1, & \text{si } i \text{ y } j \text{ con nodos adyacentes} \\ 0, & \text{caso contrario} \end{cases}$$

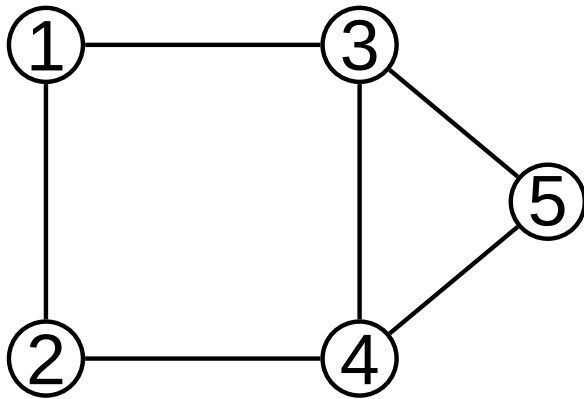
Representación computacional

Matriz de adyacencia:

Dado un grafo $G = (V, E)$, definimos su matriz de adyacencia $M(G) \in \mathcal{M}_{V \times V}$ por medio de:

$$m_{ij} := \begin{cases} 1, & \text{si } i \text{ y } j \text{ con nodos adyacentes} \\ 0, & \text{caso contrario} \end{cases}$$

Ejemplo:



$$M(G) = \begin{pmatrix} 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 & 1 \\ 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 & 0 \end{pmatrix}$$

Representación computacional

Matriz de incidencia:

Dado un grafo $G = (V, E)$, definimos su matriz de incidencia $H(G) \in \mathcal{M}_{V \times E}$ por medio de:

$$h_{ij} := \begin{cases} 1, & \text{si la arista } j \text{ es incidente al nodo } i \\ 0, & \text{caso contrario} \end{cases}$$

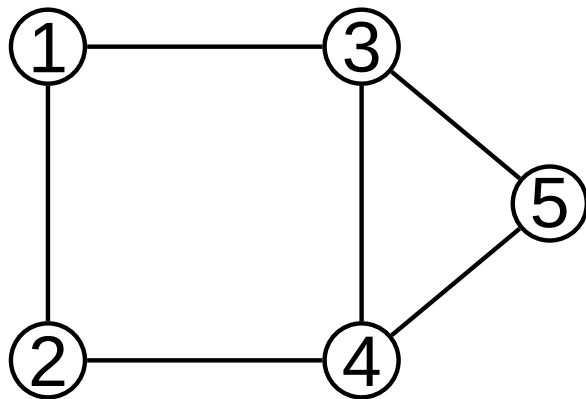
Representación computacional

Matriz de incidencia:

Dado un grafo $G = (V, E)$, definimos su matriz de incidencia $H(G) \in \mathcal{M}_{V \times E}$ por medio de:

$$h_{ij} := \begin{cases} 1, & \text{si la arista } j \text{ es incidente al nodo } i \\ 0, & \text{caso contrario} \end{cases}$$

Ejemplo:



$$H(G) = \begin{pmatrix} 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 1 \end{pmatrix}$$

Representación computacional

Listas de adyacencia:

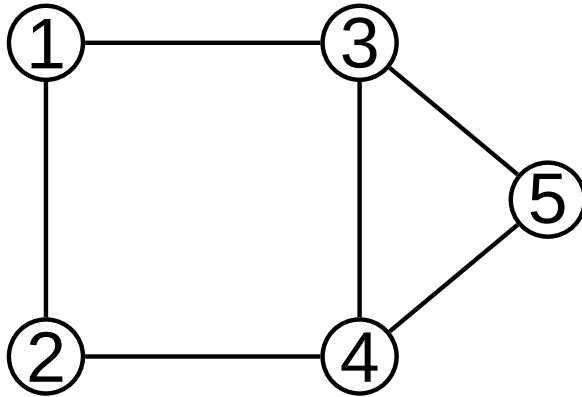
Dado un grafo $G = (V, E)$, almacenamos para cada nodo $i \in V$ una lista con su vecindad.

Representación computacional

Listas de adyacencia:

Dado un grafo $G = (V, E)$, almacenamos para cada nodo $i \in V$ una lista con su vecindad.

Ejemplo:



$$L[1] = \langle 2, 3 \rangle$$

$$L[2] = \langle 1, 4 \rangle$$

$$L[3] = \langle 1, 4, 5 \rangle$$

$$L[4] = \langle 2, 3, 5 \rangle$$

$$L[5] = \langle 3, 4 \rangle$$

Dos problemas clásicos

Problema del circuito euleriano:

Dado un grafo $G = (V, E)$, determinar si existe un **camino cerrado** que visita todas las **aristas** de G .

Dos problemas clásicos

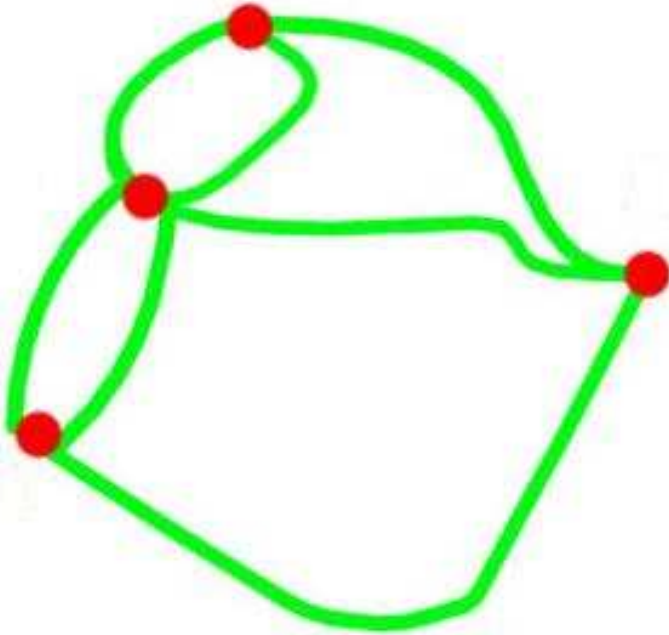
Problema del circuito euleriano:

Dado un grafo $G = (V, E)$, determinar si existe un **camino cerrado** que visita todas las **aristas** de G .

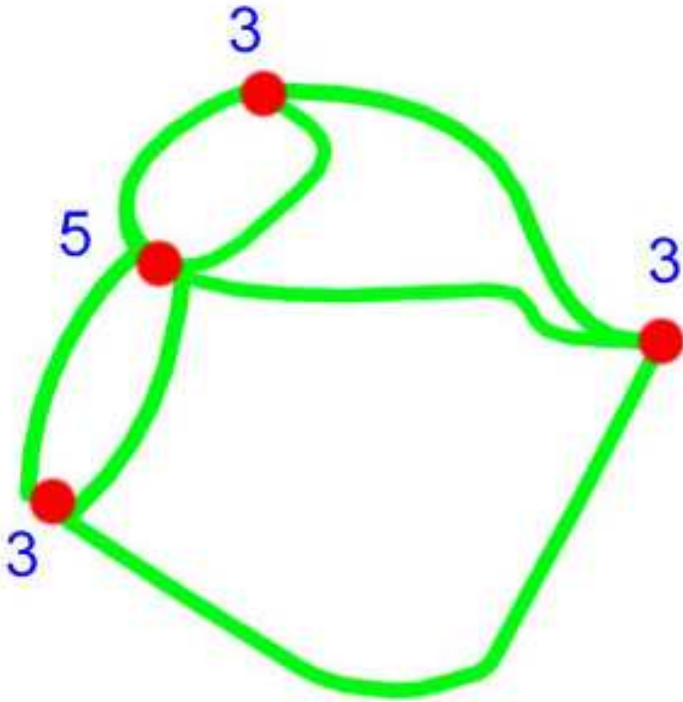
Problema del circuito hamiltoniano:

Dado un grafo $G = (V, E)$, determinar si existe un **camino cerrado** que visita todos los **nodos** de G .

Y los puentes?



Y los puentes?



Teorema:

Un grafo admite un circuito euleriano si y sólo si todos sus nodos tienen grado par.

Y los puentes?



En contraste, el problema del circuito hamiltoniano es muy **difícil**...

grado par.

sólo
enen

Matemáticas discretas

Problemas:

- Caminos eulerianos
- Caminos más cortos
- Árboles generadores
- Flujos máximos
- Programación lineal y entera
- ...

Matemáticas discretas

Problemas:

- Caminos eulerianos
- Caminos más cortos
- Árboles generadores
- Flujos máximos
- Programación lineal y entera
- ...

Aplicaciones:

- Logística
- Transporte
- Telecomunicaciones
- Redes de distribución
- Secuenciamiento genético
- ...